OFPPT
La Voie de l'Avenir

ISTA BAB TIZIMI – MEKNES
Développement Digital

Guide Pratique

Module N° 205 :
Développement Backend

Séquence N° 01 : Découvrir les framework et CMS PHP



CMS | Framework | php | Base de données

OFPPT
Ensemble pour développer les compétences

Apprendre Pratiquer • Réussir

Votre réussite notre ambition

Conception : *Said GAHI – Formateur en Développement Digital*
ISTA BAB TIZIMI – *Meknès*

Avant de commencer

1. Pourquoi ces exercices ?

Laravel repose sur trois piliers que vous devez maîtriser **avant** d'ouvrir le framework : les **fonctions**, les **tableaux** (collections, configuration, résultats de requêtes) et la **programmation orientée objet** (modèles, contrôleurs, services) les **espaces de noms**. Ces cinq travaux pratiques construisent cette base pas à pas : chaque exercice réutilise les acquis du précédent.

2. Parcours pédagogique

N°	Thème	Notions clés	Durée
Activité 1	Fonctions et calculs	function, paramètres typés, return, valeurs par défaut, fonctions mathématiques	30min
Exercice 2	Tableaux indexés	création, parcours, ajout/suppression, tri, array_map / filter / reduce	30min
Exercice 3	Tableaux associatifs	clé => valeur, tableaux multidimensionnels, usort, array_column	30min
Exercice 4	Classes simples	attributs, constructeur, getters/setters, méthodes d'instance et statiques	30min
Exercice 5	Héritage et polymorphisme	extends, parent::, classe abstraite, redéfinition, instanceof	1h

3. Organisation du projet

Créez un dossier de travail unique et respectez la structure suivante. Chaque exercice a son propre dossier, et chaque question son propre fichier ou sa propre section clairement commentée.

```
php-backend/
├── activite1-fonctions/
│   ├── fonctions.php        <- vos fonctions
│   └── test.php             <- vos appels et vérifications
├── exercice2-tableaux-indexes/
├── exercice3-tableaux-associatifs/
├── exercice4-classes/
│   └── CompteBancaire.php
└── exercice5-heritage/
    ├── CompteBancaire.php
    ├── CompteEpargne.php
    ├── CompteCourant.php
    └── index.php
```

4. Exécuter un script PHP

- **En ligne de commande** (recommandé pour ces exercices) : `php chemin/vers/test.php`
- **Serveur intégré** (pour les pages HTML) : `php -S localhost:8000` puis ouvrir `http://localhost:8000/fichier.php`
- **Vérifier la syntaxe sans exécuter** : `php -l fichier.php`

5. Règles de qualité à respecter dans tous les exercices

- Commencer chaque fichier par `<?php` puis `declare(strict_types=1);`
- Nommer en **camelCase** les variables, fonctions et méthodes (`calculerTotal`), en **PascalCase** les classes (`CompteBancaire`), en MAJUSCULES les constantes (`TAUX_TVA`).
- Typer systématiquement les paramètres et les valeurs de retour (`int`, `float`, `string`, `bool`, `array`, `void`).
- Une fonction = une responsabilité. Pas d'`echo` à l'intérieur d'une fonction de calcul : elle **retourne** une valeur.
- Commenter l'intention (le « pourquoi »), pas l'évidence.
- Indenter avec 4 espaces et nommer les fichiers de classes comme la classe (`CompteBancaire.php`), conformément à la norme **PSR-12** utilisée par Laravel.

Comment utiliser ce cahier

Chaque exercice contient : **objectifs**, **rappel de cours**, **énoncé détaillé par étapes**, **résultat attendu**, **pour aller plus loin** et une **liste de contrôle** d'auto-évaluation.

Les blocs de code intitulés « Squelette » sont à **compléter** (zones `// TODO`). Ne copiez pas aveuglément : tapez le code pour mémoriser la syntaxe.

Activité 1 — Développer des fonctions de calcul

DURÉE	NIVEAU	DOSSIER	LIVRABLE
30min	Débutant	activite1-fonctions/	fonctions.php + test.php

Objectifs

- Déclarer et appeler une fonction PHP avec des paramètres typés et un type de retour.
- Utiliser les valeurs par défaut, les paramètres et le retour de valeur.
- Mobiliser les opérateurs arithmétiques et les fonctions mathématiques natives.
- Gérer un cas d'erreur simple (division par zéro) avec une exception.
- Séparer la **logique** (fonctions) de l'**affichage** (script de test).

Rappel de cours

Syntaxe d'une fonction

```php
<?php
declare(strict_types=1);

// function nomDeLaFonction(type $parametre, type $autre = valeurParDefaut): typeRetour
function additionner(float $a, float $b): float
{
    return $a + $b;
}

echo additionner(2.5, 4);   // 6.5
```

Opérateurs et fonctions mathématiques utiles

Élément	Rôle	Exemple
`+ - * /`	Opérations de base	10 / 4 → 2.5
`%`	Reste de la division entière (modulo)	10 % 3 → 1
`**`	Puissance	2 ** 3 → 8
`intdiv(a, b)`	Division entière	intdiv(10, 3) → 3
`round(x, n)`	Arrondi à n décimales	round(3.14159, 2) → 3.14
`floor(x) / ceil(x)`	Arrondi inférieur / supérieur	floor(4.7) → 4
`abs(x)`	Valeur absolue	abs(-5) → 5
`sqrt(x)`	Racine carrée	sqrt(81) → 9
`pi() / M_PI`	Constante π	3.14159…
`max(…) / min(…)`	Plus grande / plus petite valeur	max(3, 9, 5) → 9
`number_format(x, n, ",", " ")`	Mise en forme d'un nombre	1234.5 → 1 234,50

Valeur par défaut et paramètres

```php
<?php
declare(strict_types=1);

function calculerTTC(float $ht, float $tva = 20.0): float
{
    return $ht * (1 + $tva / 100);
}

function somme(float ...$nombres): float   // nombre variable d'arguments
{
    return array_sum($nombres);
}

echo calculerTTC(100);        // 120   (TVA par défaut : 20 %)
echo calculerTTC(100, 10);    // 110
echo somme(1, 2, 3.5);        // 6.5
```

> **Attention**
>
> Avec `declare(strict_types=1)`, passer la chaîne `"5"` à un paramètre `int` provoque une `TypeError`. C'est voulu : cela évite des bugs silencieux.
>
> Une fonction qui **affiche** (`echo`) ne peut pas être réutilisée dans un autre calcul. Une fonction qui **retourne** si.

Énoncé

Étape 1 — Les quatre opérations

1. Créez `fonctions.php` et écrivez `additionner`, `soustraire`, `multiplier` et `diviser` (deux `float` en paramètres, un `float` en retour).
2. Dans `diviser`, si le diviseur vaut `0`, **lancez une exception** : `throw new InvalidArgumentException("Division par zéro impossible");`.
3. Dans `test.php`, faites `require_once __DIR__ . "/fonctions.php";` puis affichez le résultat de chaque fonction avec des phrases complètes, par exemple : `12 + 8 = 20`.
4. Testez la division par zéro dans un bloc `try { ... } catch (InvalidArgumentException $e) { echo $e->getMessage(); }`.

Étape 2 — Fonctions logiques

1. `estPair(int $n): bool` : retourne `true` si `$n` est pair (utilisez `%`).
2. `estPositif(float $n): bool`.
3. `maximum(float $a, float $b, float $c): float` **sans** utiliser `max()`, puis réécrivez-la avec `max()` et comparez.
4. `estPremier(int $n): bool` : un nombre premier n'est divisible que par 1 et par lui-même (testez les diviseurs de 2 à `floor(sqrt($n))`).

Étape 3 — Calculs commerciaux

1. `calculerTTC(float $ht, float $tva = 20.0): float`.

2. `calculerRemise(float $prix, float $pourcentage): float` : retourne le prix **après** remise, arrondi à 2 décimales.
3. `calculerFacture(float $prixUnitaire, int $quantite, float $remise = 0, float $tva = 20.0): float` : réutilise `calculerRemise` et `calculerTTC` (**appel de fonctions entre elles**). Exemple : 10 articles à 100 MAD, remise 10 %, TVA 20 % → 1 080 MAD.
4. Affichez le montant avec `number_format($montant, 2, ",", " ") . " MAD"`.

Étape 4 — Statistiques et géométrie

1. `moyenne(float ...$notes): float` : moyenne d'un nombre variable de notes (gérez le cas « aucune note » en retournant `0`).
2. `aireCercle(float $rayon): float` et `perimetreCercle(float $rayon): float` (utilisez `M_PI`, arrondi à 2 décimales).
3. `celsiusVersFahrenheit(float $c): float` avec la formule `F = C × 9/5 + 32`, et la fonction inverse.
4. `convertirDevise(float $montant, float $taux): float` (ex. MAD → EUR avec un taux de `0.092`).

Étape 5 — Fonctions à boucle

1. `factorielle(int $n): int` : `5! = 5 × 4 × 3 × 2 × 1 = 120`. Lancez une exception si `$n < 0`.
2. `fibonacci(int $n): array` : retourne les `$n` premiers termes (`0, 1, 1, 2, 3, 5, 8…`).
3. `pgcd(int $a, int $b): int` avec l'algorithme d'Euclide (`while ($b !== 0) { ... }`).
4. **Défi :** `puissance(float $x, int $n): float` **sans** utiliser `**` ni `pow()` (par boucle), puis en version **récursive**.

Squelette de départ

fonctions.php

```php
<?php
declare(strict_types=1);

function additionner(float $a, float $b): float
{
    // TODO
}

function diviser(float $a, float $b): float
{
    // TODO : lancer InvalidArgumentException si $b == 0
}

function estPair(int $n): bool
{
    // TODO
}

function factorielle(int $n): int
{
    // TODO
}
```

test.php

```php
<?php
declare(strict_types=1);
require_once __DIR__ . '/fonctions.php';

echo "12 + 8 = " . additionner(12, 8) . PHP_EOL;
echo "7 est pair ? " . (estPair(7) ? "oui" : "non") . PHP_EOL;
echo "5! = " . factorielle(5) . PHP_EOL;

try {
    echo diviser(10, 0);
} catch (InvalidArgumentException $e) {
    echo "Erreur : " . $e->getMessage() . PHP_EOL;
}
```

Résultat attendu (extrait)

```
12 + 8 = 20
7 est pair ? non
5! = 120
Erreur : Division par zéro impossible
Facture : 1 080,00 MAD
Fibonacci(8) : 0, 1, 1, 2, 3, 5, 8, 13
PGCD(48, 18) = 6
```

Pour aller plus loin

- Créez une **calculatrice en ligne de commande** : `php calculatrice.php 12 + 8` lit `$argv`, appelle la bonne fonction et affiche le résultat.
- Réalisez un **formulaire HTML** (deux champs + liste déroulante d'opérations) qui envoie les données en `POST` vers `calcul.php` ; pensez à valider avec `is_numeric()` et à protéger l'affichage avec `htmlspecialchars()`.
- Utilisez `assert()` ou écrivez une petite fonction `verifier(string $nom, mixed $attendu, mixed $obtenu): void` pour automatiser vos tests.

Liste de contrôle

- ☐ Toutes mes fonctions sont typées (paramètres **et** retour).
- ☐ Aucune fonction de calcul ne contient d'`echo`.
- ☐ La division par zéro est gérée par une exception interceptée dans `test.php`.
- ☐ `calculerFacture` appelle `calculerRemise` et `calculerTTC` au lieu de répéter leur code.
- ☐ Mon script s'exécute sans avertissement ni erreur (`php test.php`).

Exercice 2 — Manipuler les tableaux indexés

DURÉE	NIVEAU	DOSSIER	LIVRABLE
2 h 30	Débutant +	exercice2-tableaux-indexes/	tableaux.php

Objectifs

- Créer, lire, modifier et parcourir un tableau indexé (`for`, `foreach`).
- Ajouter et retirer des éléments au début, à la fin et au milieu d'un tableau.
- Rechercher, trier, extraire une portion de tableau.
- Transformer un tableau avec `array_map`, `array_filter` et `array_reduce`.
- Distinguer les fonctions qui **modifient** le tableau d'origine de celles qui **retournent** un nouveau tableau.

Rappel de cours

Un tableau indexé associe à chaque valeur un **indice numérique** commençant à **0**.

```php
<?php
$fruits = ['pomme', 'banane', 'orange'];   // syntaxe courte (recommandée)
$vide   = [];
$suite  = range(1, 5);                      // [1, 2, 3, 4, 5]

echo $fruits[0];                // pomme
$fruits[] = 'kiwi';             // ajoute à la fin
$fruits[1] = 'fraise';          // remplace l'élément d'indice 1
echo count($fruits);            // 4

foreach ($fruits as $indice => $fruit) {
    echo "$indice : $fruit\n";
}
```

Fonctions de manipulation des tableaux

Fonction	Rôle	Exemple
`count($t)`	Nombre d'éléments	count([4,5,6]) → 3
`array_push($t, $v)`	Ajoute à la **fin** (modifie $t)	array_push($t, 9) ≡ $t[] = 9
`array_pop($t)`	Retire et retourne le **dernier**	array_pop([1,2,3]) → 3
`array_unshift($t, $v)`	Ajoute au **début** (modifie $t)	array_unshift($t, 0)
`array_shift($t)`	Retire et retourne le **premier**	array_shift([1,2,3]) → 1
`array_splice($t, $i, $n, $r)`	Supprime / remplace / insère à l'indice $i (modifie $t)	array_splice($t, 2, 0, [99])
`array_slice($t, $i, $n)`	Extrait une portion (nouveau tableau)	array_slice([1,2,3,4], 1, 2) → [2,3]
`in_array($v, $t)`	Teste la présence (bool)	in_array(3, [1,2,3]) → true
`array_search($v, $t)`	Retourne l'indice de la valeur (ou false)	array_search(3, [1,2,3]) → 2

Fonction	Rôle	Exemple
`sort($t) / rsort($t)`	Tri croissant / décroissant (modifie $t, réindexe)	sort([3,1,2]) → [1,2,3]
`array_reverse($t)`	Inverse l'ordre (nouveau tableau)	array_reverse([1,2,3]) → [3,2,1]
`array_sum($t) / array_product($t)`	Somme / produit des valeurs	array_sum([1,2,3]) → 6
`min($t) / max($t)`	Plus petite / plus grande valeur	max([4,9,2]) → 9
`array_unique($t)`	Supprime les doublons	array_unique([1,1,2]) → [1,2]
`array_merge($a, $b)`	Fusionne deux tableaux	array_merge([1],[2]) → [1,2]
`array_map($fn, $t)`	Applique une fonction à chaque élément	array_map(fn($x)=>$x*2, [1,2]) → [2,4]
`array_filter($t, $fn)`	Garde les éléments qui satisfont $fn	array_filter([1,2,3,4], fn($x)=>$x>2)
`array_reduce($t, $fn, $init)`	Réduit le tableau à une seule valeur	array_reduce([1,2,3], fn($c,$x)=>$c+$x, 0) → 6
`array_values($t)`	Réindexe à partir de 0	array_values($filtre)
`array_chunk($t, $n)`	Découpe en sous-tableaux de $n éléments	array_chunk([1,2,3,4,5], 2)
`array_diff($a, $b)`	Valeurs de $a absentes de $b	array_diff([1,2,3],[2]) → [1,3]
`array_intersect($a, $b)`	Valeurs communes à $a et $b	array_intersect([1,2],[2,3]) → [2]
`array_fill($i, $n, $v)`	Remplit un tableau avec une valeur	array_fill(0, 3, 0) → [0,0,0]
`implode($sep, $t)`	Tableau → chaîne	implode(", ", ["a","b"]) → "a, b"
`explode($sep, $s)`	Chaîne → tableau	explode(",", "a,b") → ["a","b"]
`array_key_last($t)`	Dernier indice	array_key_last([5,6,7]) → 2
`shuffle($t)`	Mélange aléatoirement (modifie $t)	shuffle($cartes)

Pièges classiques

`sort`, `rsort`, `array_push`, `array_pop`, `array_shift`, `array_unshift` et `array_splice` **modifient le tableau passé en argument** et retournent autre chose (un booléen, un nombre ou l'élément retiré). Écrire `$t = sort($t);` est une erreur.

`array_filter` **conserve les indices** d'origine : le résultat peut avoir des « trous ». Enchaînez avec `array_values()` pour réindexer.

`array_search` peut retourner `0` (premier indice) : comparez toujours avec `=== false`, jamais avec `== false`.

Fonctions anonymes et fonctions fléchées

```php
<?php
$prix = [10, 20, 30];

$avecTva = array_map(fn(float $p): float => $p * 1.2, $prix);          // [12, 24, 36]
$chers   = array_values(array_filter($prix, fn($p) => $p > 15));       // [20, 30]
$total   = array_reduce($prix, fn($cumul, $p) => $cumul + $p, 0);      // 60
```

Énoncé

Dans `tableaux.php`, partez du jeu de données suivant, à **ne pas modifier** (travaillez sur des copies quand c'est nécessaire) :

```php
<?php
declare(strict_types=1);

$notes   = [12, 15, 8, 17, 9, 14, 11];
$prenoms = ['Sara', 'Youssef', 'Imane', 'Omar', 'Khadija'];
```

Étape 1 — Lecture et parcours

1. Affichez le **premier** et le **dernier** élément de `$notes` sans écrire l'indice en dur pour le dernier (`count()` ou `array_key_last()`).
2. Affichez le nombre de notes.
3. Parcourez `$notes` avec une boucle `for` puis avec `foreach` pour afficher : `Note n°1 : 12`, `Note n°2 : 15`, …
4. Parcourez `$prenoms` avec `foreach ($prenoms as $i => $prenom)` et affichez `1. Sara`, `2. Youssef`…

Étape 2 — Ajouter, retirer, insérer

1. Ajoutez la note `16` à la fin (de **deux** façons différentes).
2. Ajoutez la note `10` au début.
3. Retirez la dernière puis la première note et affichez les valeurs retirées.
4. Insérez la note `13` à l'indice `2` **sans écraser** les autres (`array_splice`).
5. Supprimez l'élément d'indice `1` avec `unset()` puis constatez le « trou » dans les indices (`print_r`). Corrigez avec `array_values()`.

Étape 3 — Recherche

1. Vérifiez si la note `17` existe avec `in_array`.
2. Retrouvez l'indice de la note `9` avec `array_search` et affichez un message adapté si elle n'existe pas.
3. Vérifiez si le prénom `Omar` figure dans `$prenoms`.

Étape 4 — Statistiques

Revenez aux données initiales (`$notes = [12, 15, 8, 17, 9, 14, 11]`) puis calculez et affichez :

1. la somme, la moyenne (arrondie à 2 décimales), la note minimale et la note maximale ;
2. le nombre de notes supérieures ou égales à 10 (**d'abord avec une boucle**, ensuite avec `array_filter` + `count`) ;
3. le pourcentage de réussite (nombre d'admis / nombre total × 100) ;
4. la **médiane** : triez une copie du tableau puis prenez la valeur centrale (ou la moyenne des deux valeurs centrales si le nombre d'éléments est pair).

Étape 5 — Tri et extraction

1. Triez les notes par ordre croissant (`sort`) puis décroissant (`rsort`) en conservant le tableau original intact (copiez-le d'abord : `$copie = $notes;`).

2. Affichez les **3 meilleures notes** avec `rsort` + `array_slice`.
3. Affichez les prénoms triés par ordre alphabétique et inversés (`sort`, `array_reverse`).

Étape 6 — Transformer avec array_map, array_filter, array_reduce

1. **Bonus** : produisez un nouveau tableau où chaque note est augmentée de 1 point, **sans dépasser 20** (`array_map` + `min`).
2. **Admis** : produisez le tableau des notes `>= 10`, réindexé.
3. **Total** : calculez la somme des notes avec `array_reduce` (sans `array_sum`).
4. **Prénoms en majuscules** : `array_map("strtoupper", $prenoms)`.
5. **Mentions** : produisez un tableau de chaînes `"12 → Passable"` avec la règle : `< 10` Ajourné, `10–13.99` Passable, `14–15.99` Assez bien, `>= 16` Bien.

Étape 7 — Chaînes, fusion et ensembles

1. Affichez les prénoms séparés par des virgules (`implode`).
2. Transformez la chaîne `"php,js,sql,php,html,js"` en tableau (`explode`), supprimez les doublons (`array_unique`) et affichez le résultat.
3. Fusionnez `["php", "sql"]` et `["js", "sql"]` avec `array_merge` puis dédoublonnez.
4. Avec `$a = ["php","js","sql"]` et `$b = ["js","html","sql"]`, affichez les éléments **communs** (`array_intersect`) et ceux qui ne sont **que dans $a** (`array_diff`).
5. Découpez `range(1, 10)` en paquets de 3 avec `array_chunk`.

Étape 8 — Mini-problème : le carnet de notes

Écrivez des **fonctions** (voir Activité 1) qui prennent un tableau de notes en paramètre :

- `moyenne(array $notes): float`
- `mediane(array $notes): float`
- `admis(array $notes, float $seuil = 10): array`
- `meilleures(array $notes, int $n = 3): array`
- `histogramme(array $notes): void` : affiche une ligne par note avec autant de `*` que de points (`str_repeat("*", $note)`).

Testez-les avec `$notes` puis avec un tableau **vide** : aucune erreur ne doit apparaître (division par zéro, `min()` d'un tableau vide…). Prévoyez ces cas.

Résultat attendu (extrait)

Avec `$notes = [12, 15, 8, 17, 9, 14, 11]` :

```
Somme : 86 | Moyenne : 12.29 | Min : 8 | Max : 17
Admis (>= 10) : 5 sur 7 (71.43 %)
Médiane : 12
3 meilleures notes : 17, 15, 14
Avec bonus : 13, 16, 9, 18, 10, 15, 12
Fusion dédoublonnée : php, sql, js
Paquets de 3 : [1,2,3] [4,5,6] [7,8,9] [10]
```

Pour aller plus loin

- Tirez au sort un prénom avec `array_rand()` et mélangez la liste avec `shuffle()` pour former des **binômes**.

- Écrivez une fonction `rotation(array $t, int $n): array` qui décale les éléments de `$n` positions.
- Écrivez `aplatir(array $matrice): array` qui transforme `[[1,2],[3,4]]` en `[1,2,3,4]` (sans `array_merge(...$m)` d'abord, puis avec).
- Parcourez un tableau à **deux dimensions** `$matrice[$ligne][$colonne]` avec deux `foreach` imbriqués et affichez-le sous forme de grille.

Liste de contrôle

☐ Je sais quelles fonctions modifient le tableau d'origine et lesquelles en retournent un nouveau.

☐ J'ai réindexé avec `array_values()` après chaque `unset()` ou `array_filter()`.

☐ Mes fonctions gèrent un tableau vide.

☐ J'ai utilisé au moins une fois `array_map`, `array_filter` et `array_reduce` avec une fonction fléchée.

☐ Mon code ne contient aucune valeur « magique » (le seuil `10` est un paramètre ou une constante).

Exercice 3 — Manipuler les tableaux associatifs

DURÉE	NIVEAU	DOSSIER	LIVRABLE
3 h	Intermédiaire	exercice3-tableaux-associatifs/	associatifs.php

Objectifs

- Créer et manipuler des tableaux **clé => valeur**.
- Tester l'existence d'une clé (`isset`, `array_key_exists`, opérateur `??`).
- Parcourir avec `foreach ($t as $cle => $valeur)` et trier par clé ou par valeur en conservant les associations.
- Représenter des données réelles sous forme de **tableaux multidimensionnels** (liste d'enregistrements).
- Extraire, regrouper et classer des données (`array_column`, `usort`).

Rappel de cours

```php
<?php
$stagiaire = [
    'nom'     => 'Alami',
    'prenom'  => 'Sara',
    'age'     => 21,
    'langages' => ['PHP', 'JavaScript'],   // une valeur peut être un tableau
];

echo $stagiaire['prenom'];            // Sara
$stagiaire['ville'] = 'Casablanca';   // ajout d'une clé
$stagiaire['age']   = 22;             // modification
unset($stagiaire['langages']);        // suppression

echo $stagiaire['email'] ?? 'non renseigné';   // valeur par défaut si la clé est absente
```

Fonctions utiles pour les tableaux associatifs

Fonction	Rôle	Remarque
`isset($t["k"])`	Clé présente **et** valeur non nulle	Retourne false si la valeur est null
`array_key_exists("k", $t)`	Clé présente (même si valeur null)	Plus précise qu'isset pour les null
`array_keys($t)`	Liste des clés	Retourne un tableau indexé
`array_values($t)`	Liste des valeurs	Réindexe à partir de 0
`array_key_first / array_key_last`	Première / dernière clé	PHP ≥ 7.3
`array_search($v, $t)`	Clé associée à une valeur	Retourne false si absente
`array_flip($t)`	Échange clés et valeurs	Les valeurs doivent être uniques
`array_combine($cles, $vals)`	Construit un tableau à partir de deux listes	Même nombre d'éléments exigé

Fonction	Rôle	Remarque
`ksort / krsort`	Tri par **clé** croissant / décroissant	Conserve les associations
`asort / arsort`	Tri par **valeur** croissant / décroissant	Conserve les associations
`usort($t, $fn)`	Tri personnalisé (réindexe)	Utiliser l'opérateur <=>
`uasort($t, $fn)`	Tri personnalisé qui **conserve les clés**	Pour tableaux associatifs
`array_column($t, "col", "idx")`	Extrait une colonne d'un tableau d'enregistrements	Très utilisé avec les résultats SQL
`array_merge($a, $b)`	Fusion (les clés texte identiques sont écrasées)	Alternative : $a + $b
`array_map / array_filter`	Transformation / filtre	array_filter conserve les clés
`array_walk($t, $fn)`	Parcourt et peut modifier par référence	function (&$v, $k) {…}
`array_slice($t, $i, $n, true)`	Portion avec préservation des clés	4e argument = true

Tableau d'enregistrements et tri personnalisé

```php
<?php
$eleves = [
    ['nom' => 'Alami',  'note' => 15],
    ['nom' => 'Benani', 'note' => 12],
    ['nom' => 'Chraibi','note' => 18],
];

usort($eleves, fn($a, $b) => $b['note'] <=> $a['note']);   // décroissant
$noms = array_column($eleves, 'nom');                       // ['Chraibi','Alami','Benani']
$parNom = array_column($eleves, 'note', 'nom');             // ['Chraibi'=>18, 'Alami'=>15,
...]

foreach ($eleves as $e) {
    printf("%-10s %5.1f\n", $e['nom'], $e['note']);
}
```

À retenir

`<=>` (opérateur « spaceship ») retourne -1, 0 ou 1 : c'est exactement ce qu'attend `usort`. Pour trier par ordre **décroissant**, inversez les opérandes (`$b <=> $a`).

Ce format « liste d'enregistrements » (`[ ['id'=>1, ...], ['id'=>2, ...] ]`) est celui que vous retrouverez dans Laravel pour les résultats de base de données et les réponses JSON.

PHP — Cahier d'exercices pratiques

Énoncé

Étape 1 — La fiche d'un stagiaire

```php
<?php
declare(strict_types=1);

$stagiaire = [
    'nom'     => 'Alami',
    'prenom'  => 'Sara',
    'age'     => 21,
    'filiere' => 'Développement digital',
    'notes'   => ['php' => 16, 'js' => 14, 'sql' => 15],
];
```

1. Affichez : `Sara ALAMI, 21 ans, filière Développement digital` (`strtoupper`).
2. Ajoutez la clé `ville` (`Casablanca`) puis modifiez `age` à `22`.
3. Affichez la clé `email` avec l'opérateur `??` et la valeur par défaut `"non renseigné"`.
4. Testez la différence entre `isset` et `array_key_exists` : ajoutez `$stagiaire["telephone"] = null;` et comparez les deux résultats.
5. Supprimez la clé `ville` avec `unset`.
6. Affichez la note de `php`, puis la **moyenne** des trois notes (`array_sum` / `count` sur `$stagiaire["notes"]`).
7. Parcourez `$stagiaire["notes"]` avec `foreach` pour afficher `php : 16/20`, `js : 14/20`…

Étape 2 — Le panier d'achat

```php
<?php
$panier = [
    'Clavier' => 250,
    'Souris'  => 90,
    'Ecran'   => 1400,
    'Casque'  => 320,
];
```

1. Affichez le **total** du panier et le nombre d'articles.
2. Affichez le produit le **plus cher** et le **moins cher** avec leur nom (`max`, `array_search`, ou `arsort` + `array_key_first`).
3. Appliquez une remise de **10 %** à tous les articles avec `array_map` : vérifiez que les clés sont conservées.
4. Gardez uniquement les articles à plus de 200 MAD (`array_filter` : constatez que les clés sont conservées).
5. Triez le panier : par nom (`ksort`), par prix croissant (`asort`), par prix décroissant (`arsort`).
6. Ajoutez le produit `Webcam => 180` seulement s'il n'existe pas déjà. Si la clé existe, **additionnez** une quantité à la place (créez pour cela un second tableau `$quantites`).
7. Affichez le panier sous forme de tableau aligné avec `printf("%-10s %8.2f MAD\n", $produit, $prix)`.

Étape 3 — Tableaux multidimensionnels : la classe de stagiaires

```php
<?php
$stagiaires = [
    ['nom' => 'Alami',   'filiere' => 'Développement',  'notes' => [16, 14, 15]],
    ['nom' => 'Benani',  'filiere' => 'Infrastructure', 'notes' => [11, 13, 12]],
    ['nom' => 'Chraibi', 'filiere' => 'Développement',  'notes' => [18, 17, 19]],
    ['nom' => 'Daoudi',  'filiere' => 'Infrastructure', 'notes' => [9, 10, 8]],
];
```

1. Ajoutez à chaque stagiaire une clé `moyenne` (calculée avec `array_sum` / `count`). Faites-le avec un `foreach` **par référence** (`foreach ($stagiaires as &$s)`) puis `unset($s);`, ou avec `array_map`.
2. Affichez la liste des noms avec `array_column($stagiaires, "nom")`.
3. Construisez avec `array_column($stagiaires, "moyenne", "nom")` un tableau `nom => moyenne`.
4. Affichez le **classement** : triez par moyenne décroissante avec `usort` et `<=>`, puis affichez `1. Chraibi (18.00)`…
5. Affichez le **meilleur** stagiaire et la **moyenne générale** de la classe.
6. Affichez uniquement les stagiaires **admis** (moyenne ≥ 10).
7. **Regroupement :** construisez `$parFiliere` de la forme `["Développement" => [stagiaire, stagiaire], "Infrastructure" => [...]]` puis affichez, pour chaque filière, l'effectif et la moyenne de la filière.
8. Ajoutez à chaque stagiaire une clé `mention` calculée par une fonction `mention(float $moyenne): string`.

> **Attention : foreach par référence**
> Après `foreach ($t as &$x)`, la variable `$x` reste une **référence** sur le dernier élément. Toujours écrire `unset($x);` juste après la boucle, sinon une boucle suivante réutilisant `$x` écrasera ce dernier élément.

Étape 4 — Mini-projet : gestion de stock

Créez un tableau `$stock` de la forme :

```php
$stock = [
    'P001' => ['designation' => 'Clavier', 'prix' => 250.0, 'quantite' => 12],
    'P002' => ['designation' => 'Souris',  'prix' => 90.0,  'quantite' => 40],
    'P003' => ['designation' => 'Ecran',   'prix' => 1400.0,'quantite' => 5],
];
```

Écrivez les fonctions suivantes (le stock est passé **par référence** quand il doit être modifié : `array &$stock`) :

Fonction	Comportement attendu
`ajouterProduit(array &$stock, string $ref, string $designation, float $prix, int $qte): bool`	Retourne false si la référence existe déjà
`retirerStock(array &$stock, string $ref, int $qte): bool`	Retourne false si la référence est inconnue ou si la quantité est insuffisante
`valeurTotale(array $stock): float`	Somme de prix × quantité
`produitsEnRupture(array $stock, int $seuil = 10): array`	Références dont la quantité est inférieure au seuil

Fonction	Comportement attendu
`afficherStock(array $stock): void`	Affichage aligné avec printf (une ligne par produit)

Dans le script de test : ajoutez un produit, tentez d'ajouter un doublon, retirez 3 claviers, tentez de retirer 100 écrans, puis affichez la valeur totale et la liste des produits en rupture.

Résultat attendu (extrait)

```
Sara ALAMI, 21 ans, filière Développement digital
Total du panier : 2060 MAD (4 articles)
Plus cher : Ecran (1400) | Moins cher : Souris (90)
Prix après remise : Clavier 225, Souris 81, Ecran 1260, Casque 288
Classement :
 1. Chraibi (18.00)
 2. Alami   (15.00)
 3. Benani  (12.00)
 4. Daoudi  (9.00)
Moyenne Développement : 16.50 | Moyenne Infrastructure : 10.50
```

Pour aller plus loin

- Transformez `$stagiaires` en JSON avec `json_encode($stagiaires, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE)` puis relisez-le avec `json_decode($json, true)`. C'est le format d'échange des API que vous construirez avec Laravel.
- Comptez les occurrences de chaque mot d'une phrase (`str_word_count`, `array_count_values`) et affichez les 3 mots les plus fréquents.
- Écrivez `grouperPar(array $liste, string $cle): array` : une fonction **générique** de regroupement réutilisable pour n'importe quelle clé.
- Générez un tableau HTML (`<table>`) à partir de `$stagiaires` dans une page `index.php`, en échappant chaque valeur avec `htmlspecialchars()`.

Liste de contrôle

☐ Je sais choisir entre `isset`, `array_key_exists` et `??`.

☐ Je sais trier un tableau associatif par clé, par valeur, et par un critère personnalisé.

☐ J'ai fait `unset($s)` après chaque `foreach` par référence.

☐ Mes fonctions de stock ne produisent aucun avertissement PHP lorsqu'une référence est inconnue.

☐ Mes données sont séparées de l'affichage (calcul dans des fonctions, `printf`/HTML à la fin).

Exercice 4 — Implémenter des classes simples : la gestion d'un compte bancaire

DURÉE	NIVEAU	DOSSIER	LIVRABLE
3 h	Intermédiaire	exercice4-classes/	CompteBancaire.php + test.php

Objectifs

- Définir une classe avec des **attributs** et des niveaux de **visibilité** (`private`, `protected`, `public`).
- Écrire un **constructeur** qui valide les données reçues.
- Implémenter des **getters** et des **setters** pour respecter l'**encapsulation**.
- Écrire des **méthodes d'instance** (qui agissent sur `$this`) et des **membres de classe** (`static`, `const`).
- Instancier des objets, les faire collaborer et gérer les erreurs avec des **exceptions**.

Rappel de cours

Vocabulaire

Terme	Définition	Exemple
Classe	Plan de construction : regroupe attributs et méthodes	class Produit { … }
Objet / instance	Exemplaire créé à partir d'une classe	$p = new Produit("Clavier", 250);
Attribut (propriété)	Variable appartenant à l'objet	private float $prix;
Méthode	Fonction appartenant à la classe	public function afficher(): string
Constructeur	Méthode appelée à la création de l'objet	public function __construct(…)
$this	L'objet courant (dans une méthode d'instance)	$this->prix
static / self::	Membre qui appartient à la **classe**, partagé par toutes les instances	self::$compteur++
const	Constante de classe	self::TVA / Produit::TVA

Visibilité et encapsulation

Mot-clé	Accessible depuis…
`public`	N'importe où
`protected`	La classe elle-même et ses classes filles
`private`	Uniquement la classe elle-même

Principe : les attributs sont **privés** ; on y accède par des **getters** (`getPrix()`) et on les modifie par des **setters** (`setPrix()`) qui **valident** la valeur. Un objet ne doit jamais pouvoir se retrouver dans un état invalide.

Exemple commenté

```php
<?php
declare(strict_types=1);

class Produit
{
    public const TVA = 20.0;              // constante de classe
    private static int $nombre = 0;       // attribut de classe (partagé)

    private string $nom;
    private float $prix;

    public function __construct(string $nom, float $prix)
    {
        $this->nom = $nom;
        $this->setPrix($prix);            // on réutilise le setter pour valider
        self::$nombre++;
    }

    public function getNom(): string { return $this->nom; }
    public function getPrix(): float { return $this->prix; }

    public function setPrix(float $prix): void
    {
        if ($prix < 0) {
            throw new InvalidArgumentException("Le prix ne peut pas être négatif.");
        }
        $this->prix = $prix;
    }

    public function prixTtc(): float      // méthode d'instance
    {
        return $this->prix * (1 + self::TVA / 100);
    }

    public static function getNombre(): int   // méthode de classe
    {
        return self::$nombre;
    }
}

$p = new Produit('Clavier', 250);
echo $p->prixTtc();                  // 300
echo Produit::getNombre();           // 1
echo Produit::TVA;                   // 20
```

Raccourci PHP 8 : promotion de propriétés du constructeur

Au lieu de déclarer les attributs puis de les affecter un par un, on peut écrire : `public function __construct(private string $nom, private float $prix = 0.0) {}`. Les attributs sont déclarés et initialisés automatiquement.

Dans cet exercice, utilisez d'abord la **version longue** pour bien comprendre le mécanisme ; vous pourrez ensuite refactoriser.

Énoncé

Étape 1 — Conception (sur papier, 10 minutes)

Complétez le diagramme de classe ci-dessous. Le signe `-` désigne un membre privé, `+` un membre public, et le soulignement un membre statique.

CompteBancaire	
Attributs	- numero : string | - titulaire : string | - solde : float | - nombreComptes : int (static)
Constantes	+ DEVISE = "MAD" | + PLAFOND_RETRAIT = 5000.0
Constructeur	+ __construct(numero, titulaire, soldeInitial = 0.0)
Getters / setter	+ getNumero() + getTitulaire() + getSolde() + setTitulaire(titulaire)
Méthodes d'instance	+ deposer(montant) : void + retirer(montant) : void + virerVers(compte, montant) : void + __toString() : string
Méthodes de classe	+ getNombreComptes() : int + estNumeroValide(numero) : bool

> **Question de réflexion**
> Pourquoi **n'y a-t-il pas** de `setSolde()` ni de `setNumero()` ? Notez votre réponse en commentaire dans le code. (Indice : qui doit être autorisé à modifier un solde ? Un numéro de compte peut-il changer ?)

Étape 2 — La classe CompteBancaire

Créez `CompteBancaire.php`. Respectez les règles de gestion suivantes :

Élément	Règle de gestion
`Constructeur`	Reçoit le numéro, le titulaire et le solde initial (0 par défaut). Refuse un numéro invalide (format `CB` + 6 chiffres, ex. `CB000123`) et un solde initial négatif en lançant une `InvalidArgumentException`. Incrémente le compteur de comptes.
`setTitulaire`	Supprime les espaces superflus (`trim`) et refuse un nom vide.
`deposer`	Le montant doit être strictement positif. Augmente le solde.
`retirer`	Le montant doit être strictement positif, **inférieur ou égal au solde**, et **inférieur ou égal à `PLAFOND_RETRAIT`**. Sinon : exception (`RuntimeException` pour solde insuffisant ou plafond dépassé, `InvalidArgumentException` pour montant invalide).
`virerVers`	Retire du compte courant et dépose sur le compte destinataire (réutilisez `retirer` et `deposer`). Refuse de virer vers soi-même.
`__toString`	Retourne par exemple : `CB000001 - Sara Alami : 1300.00 MAD`.
`getNombreComptes`	Méthode **statique** : nombre de comptes créés depuis le début du script.
`estNumeroValide`	Méthode **statique** : teste le format avec `preg_match("/^CB\d{6}$/", $numero) === 1`.

Squelette

`CompteBancaire.php`

```php
<?php
```

```php
declare(strict_types=1);

class CompteBancaire
{
    public const DEVISE = 'MAD';
    public const PLAFOND_RETRAIT = 5000.0;

    private static int $nombreComptes = 0;

    private string $numero;
    private string $titulaire;
    private float $solde;

    public function __construct(string $numero, string $titulaire, float $soldeInitial = 0.0)
    {
        // TODO 1 : valider le numéro (self::estNumeroValide) et le solde initial
        // TODO 2 : affecter les attributs ($this->setTitulaire pour valider le nom)
        // TODO 3 : incrémenter self::$nombreComptes
    }

    // --- Getters / setter ---
    public function getNumero(): string { /* TODO */ }
    public function getTitulaire(): string { /* TODO */ }
    public function getSolde(): float { /* TODO */ }
    public function setTitulaire(string $titulaire): void { /* TODO */ }

    // --- Méthodes d'instance ---
    public function deposer(float $montant): void { /* TODO */ }
    public function retirer(float $montant): void { /* TODO */ }
    public function virerVers(CompteBancaire $destinataire, float $montant): void { /* TODO */
}
    public function __toString(): string { /* TODO */ }

    // --- Méthodes de classe ---
    public static function getNombreComptes(): int { /* TODO */ }
    public static function estNumeroValide(string $numero): bool { /* TODO */ }
}
```

Étape 3 — Le script de test

Dans `test.php`, après `require_once "CompteBancaire.php";`, réalisez le scénario suivant en **interceptant chaque erreur** avec `try / catch` pour que le script continue :

1. Créez `$c1` (`CB000001`, `Sara Alami`, 1000) et `$c2` (`CB000002`, `Omar Benani`, solde initial par défaut).
2. Déposez 500 sur `$c1`, retirez 200, puis virez 300 vers `$c2`. Affichez les deux comptes.
3. Tentez de retirer 5000 sur `$c1` (solde insuffisant), de déposer -50 (montant invalide) et de créer un compte avec le numéro `"XYZ"`.
4. Modifiez le titulaire de `$c2` avec `setTitulaire("  Omar BENANI  ")` et affichez le résultat (espaces supprimés).
5. Tentez de modifier directement `$c1->solde = 1000000;` : lisez le message d'erreur PHP et **expliquez-le** en commentaire.
6. Affichez `CompteBancaire::getNombreComptes()` et `CompteBancaire::DEVISE`.

Résultat attendu

```
CB000001 - Sara Alami : 1000.00 MAD
CB000002 - Omar Benani : 300.00 MAD
Erreur : Solde insuffisant.
Erreur : Le montant doit être strictement positif.
Erreur : Numéro de compte invalide : XYZ
CB000002 - Omar BENANI : 300.00 MAD
Fatal error: Cannot access private property CompteBancaire::$solde
Nombre de comptes créés : 2 (devise : MAD)
```

(Les messages exacts sont libres ; l'important est la logique. L'erreur sur la propriété privée arrête le script : placez-la en dernier ou entourez-la d'un `try / catch (Error $e)`.)

Pour aller plus loin

- **Historique** : ajoutez un attribut `private array $operations = [];` où chaque dépôt/retrait enregistre un tableau `["type" => "depot", "montant" => 500, "date" => date("Y-m-d H:i")]`, puis une méthode `getReleve(): array`.
- **Immutabilité** : déclarez `numero` en `readonly` (PHP ≥ 8.1) et observez ce qui se passe si vous tentez de le modifier.
- **Promotion de propriétés** : réécrivez le constructeur avec la syntaxe PHP 8 abrégée.
- **Exceptions métier** : créez `class SoldeInsuffisantException extends RuntimeException {}` et utilisez-la dans `retirer()`.
- **Compteur de classe** : ajoutez `static getTotalDepots()` qui additionne les soldes de tous les comptes (indice : un attribut statique mis à jour dans `deposer`/`retirer`).

Liste de contrôle

☐ Tous les attributs sont `private`, aucun accès direct depuis l'extérieur.

☐ Il n'existe aucun moyen de mettre le solde à une valeur invalide.

☐ Le constructeur réutilise le setter pour valider le titulaire (pas de code dupliqué).

☐ `virerVers` réutilise `retirer` et `deposer`.

☐ Je sais expliquer la différence entre `$this->`, `self::` et `NomDeClasse::`.

☐ Je sais pourquoi `getNombreComptes()` est statique alors que `getSolde()` ne l'est pas.

Exercice 5 — Héritage et polymorphisme

DURÉE	NIVEAU	DOSSIER	LIVRABLE
3 h 30	Intermédiaire +	exercice5-heritage/	4 classes + index.php

Objectifs

- Créer une **hiérarchie de classes** avec `extends` et réutiliser le code d'une classe mère.
- Utiliser `protected`, `parent::__construct()` et `parent::méthode()`.
- Définir une **classe abstraite** et des **méthodes abstraites** que les classes filles doivent implémenter.
- **Redéfinir** (override) une méthode pour changer son comportement dans une classe fille.
- Mettre en œuvre le **polymorphisme** : manipuler des objets de types différents via le type de la classe mère.
- Utiliser `instanceof`, la **composition** (une classe `Banque` qui contient des comptes) et les **types** dans les signatures.

Rappel de cours

Héritage

Une classe **fille** hérite des attributs et méthodes **non privés** de sa classe **mère** et peut en ajouter ou en redéfinir. PHP n'autorise **qu'une seule** classe mère (héritage simple).

```php
<?php
declare(strict_types=1);

abstract class Animal
{
    public function __construct(protected string $nom) {}

    abstract public function parler(): string;       // à implémenter dans chaque fille

    public function presenter(): string               // méthode commune, héritée telle quelle
    {
        return "{$this->nom} dit : " . $this->parler();
    }
}

class Chien extends Animal
{
    public function parler(): string { return 'Wouaf'; }
}

class Chat extends Animal
{
    public function __construct(string $nom, private bool $domestique = true)
    {
        parent::__construct($nom);                    // appel du constructeur de la mère
    }

    public function parler(): string { return 'Miaou'; }

    public function presenter(): string               // redéfinition + réutilisation
```

```
    {
        return parent::presenter() . ($this->domestique ? ' (chat domestique)' : '');
    }
}

// POLYMORPHISME : même appel, comportement différent selon le type réel de l'objet
$animaux = [new Chien('Rex'), new Chat('Mimi')];
foreach ($animaux as $a) {
    echo $a->presenter() . PHP_EOL;
}
// Rex dit : Wouaf
// Mimi dit : Miaou (chat domestique)

var_dump($animaux[1] instanceof Animal);   // true
// new Animal('X');                         // Erreur : classe abstraite non instanciable
```

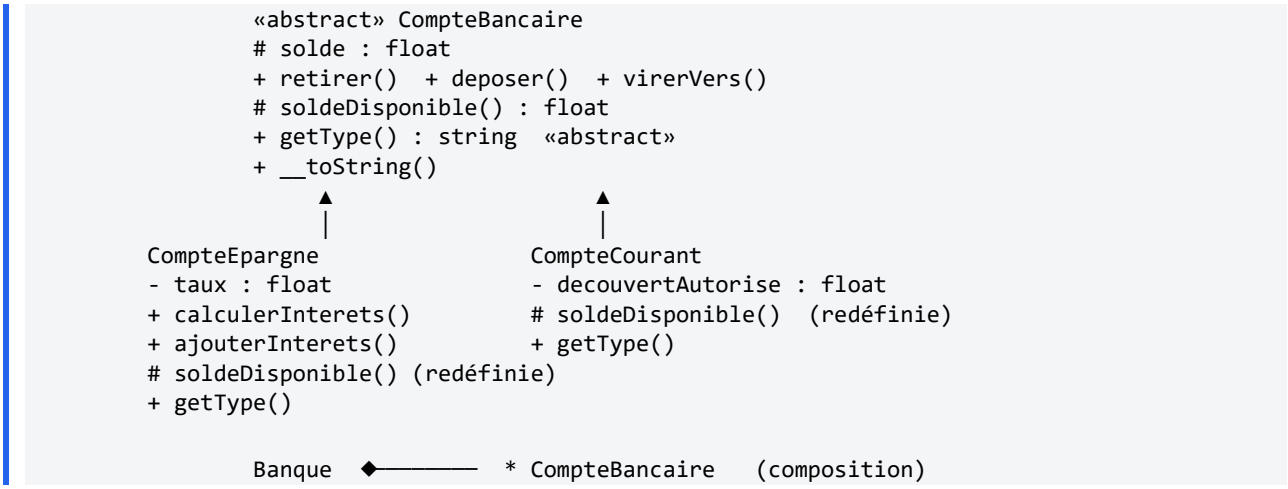
Mots-clés à retenir

Mot-clé	Effet
`extends`	La classe hérite d'une autre classe
`parent::`	Accède à la version de la classe mère (constructeur ou méthode redéfinie)
`protected`	Visible dans la classe et ses filles, invisible à l'extérieur
`abstract class`	Ne peut pas être instanciée ; sert de modèle commun
`abstract function`	Signature sans corps : **obligation** pour chaque classe fille concrète de l'implémenter
`final`	Interdit l'héritage de la classe (ou la redéfinition de la méthode)
`instanceof`	Teste si un objet est une instance d'une classe ou de l'un de ses parents
`static::` vs `self::`	`self::` désigne la classe où le code est écrit ; `static::` la classe réellement appelée (liaison tardive)

> **Héritage ou composition ?**
> On utilise l'héritage pour exprimer « **est un** » (un compte épargne **est un** compte bancaire). On utilise la composition pour « **a un** » (une banque **a des** comptes).
> Règle pratique : si la phrase « X est un Y » est fausse, n'utilisez pas `extends`.

Énoncé

On reprend le domaine bancaire de l'Exercice 4 pour construire la hiérarchie suivante :



Étape 1 — Refactoriser la classe mère

1. Copiez `CompteBancaire.php` de l'Exercice 4 dans le dossier de l'Exercice 5.
2. Transformez la classe en **classe abstraite** : `abstract class CompteBancaire`.
3. Passez l'attribut `$solde` en **`protected`** (les filles en auront besoin). Les autres attributs restent `private` avec leurs getters.
4. Ajoutez la méthode abstraite `abstract public function getType(): string;`.
5. Ajoutez la méthode `protected function soldeDisponible(): float` qui retourne par défaut `$this->solde` : c'est le montant maximal que l'on peut retirer.
6. Modifiez `retirer()` pour que la règle « solde suffisant » compare le montant à `$this->soldeDisponible()` **au lieu de** `$this->solde`. Le plafond de retrait et le contrôle du montant positif restent inchangés.
7. Modifiez `__toString()` pour inclure le type : `[Épargne] CB000001 - Sara Alami : 2000.00 MAD`.

> **Pourquoi cette conception ?**
> La classe mère décrit **l'algorithme** du retrait (vérifications, mise à jour du solde) une seule fois. Chaque classe fille ne redéfinit que ce qui **varie** : la notion de « solde disponible ». C'est ce qu'on appelle le patron de méthode (*template method*), très utilisé dans les frameworks comme Laravel.

Étape 2 — La classe CompteEpargne

1. Créez `CompteEpargne extends CompteBancaire` avec l'attribut `private float $taux` (en %) et la constante `SOLDE_MINIMUM = 100.0`.
2. Constructeur : `(string $numero, string $titulaire, float $soldeInitial, float $taux)`. Il appelle `parent::__construct(...)`, puis valide que `$taux` est compris entre 0 et 100 (exception sinon).
3. Redéfinissez `soldeDisponible()` : retourne `max(0, $this->solde - self::SOLDE_MINIMUM)`. Un compte épargne doit toujours conserver 100 MAD.
4. Implémentez `getType()` : retourne `"Épargne"`.
5. Ajoutez `calculerInterets(): float` (`solde × taux / 100`) et `ajouterInterets(): void` qui dépose les intérêts via `deposer()`.
6. Redéfinissez `__toString()` : `parent::__toString()` suivi de `(taux 3 %)`.

Étape 3 — La classe CompteCourant

1. Créez `CompteCourant extends CompteBancaire` avec `private float $decouvertAutorise`.
2. Constructeur : `(numero, titulaire, soldeInitial, decouvertAutorise = 0.0)`. Refusez un découvert négatif.
3. Redéfinissez `soldeDisponible()` : retourne `$this->solde + $this->decouvertAutorise`. Le solde peut donc devenir négatif, mais jamais au-delà du découvert autorisé.
4. Implémentez `getType()` : retourne `"Courant"`.
5. Redéfinissez `__toString()` pour ajouter `(découvert autorisé 1000.00 MAD)`.
6. Ajoutez `estEnDecouvert(): bool` (retourne `true` si le solde est négatif).

Étape 4 — Démontrer le polymorphisme

Dans `index.php`, chargez les trois classes avec `require_once` (la classe mère **en premier**), puis :

1. Créez un `CompteEpargne` (`CB000001`, `Sara Alami`, 2000, taux 3) et un `CompteCourant` (`CB000002`, `Omar Benani`, 800, découvert 1000).
2. Essayez de faire `new CompteBancaire(...)` : constatez l'erreur « Cannot instantiate abstract class ».
3. Placez les deux comptes dans un tableau `$comptes` et **parcourez-le avec un seul** `foreach` : affichez chaque compte (`echo $compte`).
4. Créez une fonction `tenterRetrait(CompteBancaire $compte, float $montant): void` qui essaie de retirer, affiche « accepté » ou « refusé » (interception de l'exception) et le nouveau solde. **Le paramètre est typé avec la classe mère** : la fonction accepte n'importe quel type de compte, présent ou futur.
5. Appelez `tenterRetrait` avec 1700 puis avec 300 sur chacun des comptes, **dans la même boucle**. Observez que le même appel produit des résultats différents selon le type réel de l'objet.
6. Appelez `ajouterInterets()` sur le compte épargne : utilisez `instanceof CompteEpargne` dans la boucle pour ne l'appeler que sur les comptes concernés.

Étape 5 — La classe Banque (composition)

1. Créez la classe `Banque` avec un attribut privé `array $comptes = []` (indexés par numéro de compte) et un nom.
2. `ajouterCompte(CompteBancaire $compte): void` : refuse un numéro déjà présent.
3. `trouverCompte(string $numero): ?CompteBancaire` : retourne `null` si inconnu (type de retour **nullable**).
4. `totalDepots(): float` : somme des soldes de tous les comptes.
5. `crediterInterets(): int` : crédite les intérêts de tous les comptes épargne (avec `instanceof`) et retourne le nombre de comptes concernés.
6. `afficherComptes(): void` : affiche tous les comptes, **un par ligne**, sans se soucier de leur type (polymorphisme sur `__toString` et `getType`).
7. `comptesParType(): array` : retourne un tableau associatif `["Épargne" => 2, "Courant" => 1]` (utilisez `getType()` comme clé).

Résultat attendu (scénario d'index.php)

```
[Épargne] CB000001 - Sara Alami : 2000.00 MAD (taux 3 %)
[Courant] CB000002 - Omar Benani : 800.00 MAD (découvert autorisé 1000.00 MAD)
Intérêts crédités : 60.00 MAD -> nouveau solde : 2060.00 MAD
Retrait de 1700 : Épargne accepté (360.00) | Courant accepté (-900.00)
Retrait de 300  : Épargne refusé (360.00)  | Courant refusé (-900.00)
Comptes par type : Épargne = 1, Courant = 1
Nombre de comptes créés : 2
```

Vérifiez vos calculs

Épargne : solde 2060 − 100 (minimum) = 1960 disponibles ; après un retrait de 1700 il reste 360, donc 260 disponibles : un retrait de 300 est refusé.

Courant : 800 + 1000 = 1800 disponibles ; après un retrait de 1700 le solde est −900, il reste 100 disponibles : un retrait de 300 est refusé.

Pour aller plus loin

- **Nouvelle classe** : créez `ComptePro extends CompteCourant` avec des frais de 1 MAD par retrait (redéfinissez `retirer()` en appelant `parent::retirer()`). Vérifiez que `Banque` et `tenterRetrait` fonctionnent **sans aucune modification** : c'est la puissance du polymorphisme.
- **Interface** : définissez `interface Imposable { public function calculerImpot(): float; }` et implémentez-la uniquement dans `CompteEpargne` (15 % des intérêts). Une classe peut implémenter plusieurs interfaces alors qu'elle n'hérite que d'une classe.
- **final** : déclarez `final public function virerVers(...)` dans la classe mère et vérifiez qu'aucune fille ne peut la redéfinir.
- **Tri polymorphe** : triez les comptes de la banque par solde décroissant avec `usort` et affichez-les.
- **Autoload** : remplacez les `require_once` par `spl_autoload_register(function (string $classe) { require_once __DIR__ . "/{$classe}.php"; });`. C'est le principe d'autoload que vous retrouverez avec Composer et Laravel.

Liste de contrôle

- ☐ La classe `CompteBancaire` est abstraite et ne peut pas être instanciée.
- ☐ Les classes filles ne dupliquent pas le code de `retirer()` : elles ne redéfinissent que `soldeDisponible()`.
- ☐ J'appelle `parent::__construct()` dans les constructeurs des classes filles.
- ☐ `tenterRetrait` est typée avec la classe mère et fonctionne avec toutes les sous-classes.
- ☐ Je sais expliquer la différence entre **redéfinition** (override) et **surcharge** (PHP ne connaît pas la surcharge classique).
- ☐ Je sais justifier le choix héritage / composition dans ce modèle.

Annexe A — Erreurs fréquentes et messages PHP

Message ou symptôme	Cause probable	Correction
Parse error: syntax error, unexpected …	Point-virgule, accolade ou parenthèse manquant(e)	Regarder la ligne indiquée **et la précédente** ; `php -l fichier.php`
Warning: Undefined variable / array key	Variable ou clé inexistante	Initialiser, ou utiliser `??` / `isset`
TypeError: … must be of type int, string given	`strict_types=1` et mauvais type d'argument	Convertir (`(int)`) ou corriger l'appel
DivisionByZeroError	Division ou modulo par zéro	Tester le diviseur avant
Cannot access private property	Accès direct à un attribut `private`	Passer par le getter / setter
Cannot instantiate abstract class	`new` sur une classe abstraite	Instancier une classe fille concrète
Class "X" not found	`require` oublié ou mauvais ordre (mère avant fille)	Vérifier les `require_once` et le nom du fichier
Call to undefined method	Faute de frappe ou méthode absente	Vérifier le nom, la classe réelle de l'objet (`get_class`)
Résultat de sort() = true / 1	Écriture `$t = sort($t)`	Appeler `sort($t);` sans réaffectation
Tableau avec des « trous » d'indices	`unset` ou `array_filter`	`array_values()`
Accents illisibles dans la console	Fichier non enregistré en UTF-8	Enregistrer en UTF-8 sans BOM

Fonctions de débogage

Fonction	Usage
`var_dump($x)`	Type **et** valeur (idéal pour comprendre un bug de type)
`print_r($t)`	Affichage lisible d'un tableau ou d'un objet ; ajouter `, true` pour récupérer la chaîne
`echo "<pre>" . print_r($t, true) . "</pre>";`	Même chose, lisible dans un navigateur
`get_class($obj)` / `$obj::class`	Nom de la classe réelle d'un objet
`error_reporting(E_ALL); ini_set("display_errors", "1");`	Afficher toutes les erreurs en développement

Annexe B — Grille d'évaluation formative

Barème indicatif sur **20 points** pour l'ensemble du cahier. Chaque exercice peut aussi être évalué séparément en proportion.

Critère	Activité 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5
Fonctionnement correct (résultats conformes)	2	2	2	2	2
Respect des consignes et complétude	1	1	1	1	1
Qualité du code (nommage, typage, PSR-12)	0,5	0,5	0,5	0,5	0,5
Gestion des erreurs et des cas limites	0,5	0,5	0,5	0,5	0,5
Maîtrise de la notion clé (voir ci-dessous)	—	—	—	—	—
Sous-total par exercice	4	4	4	4	4

Notion clé évaluée à l'oral (4 points au total, 1 point par question posée lors de la correction) : *retour de valeur vs affichage* ; *fonctions qui modifient un tableau vs qui en retournent un* ; *encapsulation et rôle des setters* ; *polymorphisme et classe abstraite*.

Niveaux de maîtrise

Niveau	Description
Insuffisant	Le code ne s'exécute pas ou ne produit pas le résultat attendu ; notions non comprises.
Fragile	Le résultat est correct sur les cas simples ; cas limites non traités ; code dupliqué.
Satisfaisant	Résultats corrects, code typé et lisible, erreurs gérées, notions expliquées.
Très bien	En plus : réutilisation du code, bonnes pratiques spontanées, exercices « pour aller plus loin » réalisés.

Et après ? Le pont vers Laravel

Ce que vous venez de pratiquer	Où vous le retrouverez dans Laravel
Fonctions typées, valeurs par défaut	Helpers, méthodes de contrôleurs, règles de validation
Tableaux indexés (map / filter / reduce)	Collections Laravel : `collect($t)->map()->filter()->sum()`
Tableaux associatifs et multidimensionnels	Fichiers de configuration, requêtes `$request->all()`, réponses JSON, résultats Eloquent
Classes, getters/setters, static, const	Modèles Eloquent, services, constantes de configuration
Héritage, classes abstraites	`class Produit extends Model`, `class Controller`, `Migration`
Polymorphisme, interfaces, composition	Contrats, injection de dépendances, conteneur de services
Autoload, namespaces	PSR-4 et Composer